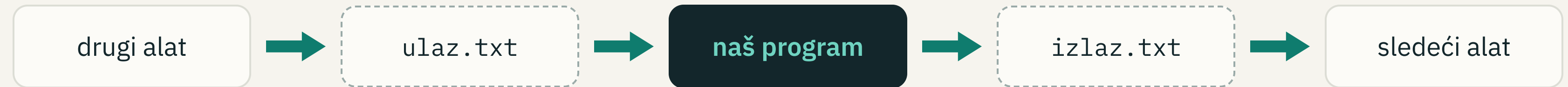


03

Čitanje i pisanje fajlova

Podaci žive u fajlovima, a naši programi rade u lancu sa drugim alatima

Zašto fajlove učimo tako rano?



Podaci su u fajlovima

- DNK i proteinske sekvence, npr. FASTA
- očitavanja sekvenciranja i njihovi kvaliteti
- SNP-ovi, filogenetska stabla, mape očitavanja
- podaci o uzorcima, matrice genetskih rastojanja

Kopiranje u kod nije rešenje

- prave sekvence su mnogo duže od onih u vežbama
- podaci ne treba da budu pomešani sa kodom
- u lancu alata podaci stižu i odlaze kroz fajlove

Tekstualni i binarni fajlovi

Većina bioloških podataka čuva se u tekstualnim fajlovima: sekvence, očitavanja i njihovi kvaliteti, SNP-ovi, filogenetska stabla, matrice genetskih rastojanja.

Tekstualni: karakteri i linije

- FASTA fajlovi sa DNK i proteinskim sekvencama
- FASTQ fajlovi sa očitavanjima sekvenciranja
- Izlaz programa komandne linije, npr. BLAST
- HTML stranice
- Python kod

Binarni: bajtovi

- Slike (JPEG, PNG)
- Audio i video fajlovi
- Kompresovani fajlovi (ZIP)

Brz test: otvorite fajl u tekst editoru. Ako se prikaže bez problema, tekstualni je, čak i ako sadržaj ne razumete.

Otvaranje i čitanje: open i read

```
my_file_name = "dna.txt"
my_file = open(my_file_name)
my_file_contents = my_file.read()
print(my_file_contents)
```

Dve česte greške

`my_file_name.read()` → `AttributeError`

read radi samo na fajl objektu, ne na imenu fajla

`print(my_file)` → `<_io.TextIOWrapper ...>`

ispisan je fajl objekat, a ne sadržaj fajla

my_file_name

string: ime fajla na disku

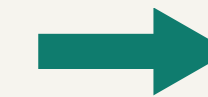
`open()`



my_file

fajl objekat: predstavlja sam
fajl

`.read()`



my_file_contents

string: tekst iz fajla

Fajl objekat: novi tip podataka

Sa fajlom radimo preko metoda

```
# string i broj koristimo direktno
my_string = 'abcdefg'
print(my_string)
my_number = 42
print(my_number + 1)
# fajl objekat: preko metoda
my_file = open("dna.txt")
print(my_file.read())
```

String i broj predstavljaju jednu vrednost. Fajl objekat predstavlja fajl na disku.

Imena su proizvoljna, ime fajla nije

```
apple = "dna.txt"
banana = open(apple)
grape = banana.read()
```

Radi isto kao `my_file_name`, `my_file` i `my_file_contents`, samo je teže za čitanje.

Ali `"dna.txt"` mora biti ime stvarnog fajla na disku.

Novi red na kraju fajla: rstrip

```
my_file = open("dna.txt")
my_dna = my_file.read()
dna_length = len(my_dna)
print("sequence is " + my_dna + " and length is " + str(dna_length))
```

IZLAZ

```
sequence is ACTGTACGTGCACTGATC
and length is 19
```

Izlaz je u dva reda, a dužina je 19, ne 18: `read` je vratio i znak za novi red sa kraja fajla, `'ACTGTACGTGCACTGATC\n'`.

```
# ukloni novi red sa kraja
my_dna = my_file.read().rstrip("\n")
```

IZLAZ

```
sequence is ACTGTACGTGCACTGATC and length is 18
```

Lanac metoda i nepostojeći fajl

Čita se s leva nadesno

```
my_dna = my_file.read().rstrip("\n")
```

`my_file` fajl objekat

`.read()` vraća string, sa `\n` na kraju

`.rstrip("\n")` poziva se na tom stringu i vraća ga bez `\n`

Ako je teško, isto se piše u dva koraka:

```
my_file_contents = my_file.read()
my_dna = my_file_contents.rstrip("\n")
```

Fajl koji ne postoji

```
my_file = open("nonexistent.txt")
```

`FileNotFoundError: [Errno 2] No such file or directory: 'nonexistent.txt'`

Kako da proverimo da li fajl postoji pre otvaranja, vidimo u poglavlju 9.

Zašto izlaz pisati u fajl?

Ispis na ekran

Odličan za kratke programe, isprobavanje i poruke o napretku. Za veliki izlaz nije:

- terminal može da izgubi početak dugog izlaza
- teško se pretražuje, a duge linije se prelamaju
- sve završi na jednom mestu
- nestaje kad se terminal zatvori

Pisanje u fajl

- rezultat ostaje i posle kraja programa, što je važno za programe koji rade satima
- različiti delovi izlaza mogu u različite fajlove
- fajl dalje čita sledeći program u lancu
- fajl se otvara u editoru i lako pretražuje

Pisanje u fajl i zatvaranje

Režim, drugi argument open

Režim	Značenje
"r"	čitanje (podrazumevano)
"w"	pisanje: postojeći sadržaj se briše
"a"	dodavanje na kraj fajla

Ako fajl ne postoji, i "w" i "a" prave novi.

`write` radi kao `print`, ali piše u fajl: na ekranu se ne vidi ništa, pa rezultat proverite u editoru.

write i close

```
my_file = open("out.txt", "w")
my_file.write("Hello world")
# write prima bilo koji string
my_file.write(str(len('AGTGCTAG')))
my_file.write("ATGC".lower())
# na kraju zatvorite fajl
my_file.close()
```

`close` je metod (a `open` funkcija): ne prima argumente i ne vraća korisnu vrednost.

Razlika između "w" i "a"

Režim "w": prepisuje

OUT.TXT PRE
stari sadržaj



```
f = open("out.txt", "w")  
f.write("novi red\n")  
f.close()
```



OUT.TXT POSLE
novi red

Režim "a": dodaje na kraj

OUT.TXT PRE
stari sadržaj



```
f = open("out.txt", "a")  
f.write("novi red\n")  
f.close()
```



OUT.TXT POSLE
stari sadržaj
novi red

Putanje do fajlova u drugim folderima

```
open("/home/martin/myfolder/myfile.txt")    # Linux
open("/Users/martin/Desktop/myfolder/myfile.txt") # macOS
open(r"c:\windows\Desktop\myfolder\myfile.txt") # Windows: r pre navodnika
```

Bez putanje

`open("dna.txt")` traži fajl u folderu iz kog je program pokrenut. Zato je najlakše da program i podaci budu u istom folderu.

Zašto r na Windowsu?

Obrnuta kosa crta počinje specijalni karakter: u `"c:\new\test.txt"` kriju se `\n` i `\t`. Slovo r to isključuje.

FASTA format

```
>sequence_one
ATCGATCGATCGATCGAT
>sequence_two
ACTAGCTAGCTAGCATCG
>sequence_three
ACTGCATCGATCGTACCT
```

Linija koja počinje sa > je zaglavlje: ime sekvence, često pristupni broj iz javne baze. Ispod je sekvenca. Jedan fajl može da sadrži više sekvenci.

Vežba: FASTA fajl sa tri sekvence

Zaglavlje	DNK sekvenca
ABC123	ATCGTACGATCGATCGCTAGACGTATCG
DEF456	actgatcgacgatcgatcgatcacgact
HIJ789	ACTGAC-ACTGT--ACTGTA----CATGTG

Sve sekvence velikim slovima, samo baze A, T, G i C.

Rešenje gradimo korak po korak na sledećim slajdovima, kao u knjizi.

Korak 1: prvo ispis na ekran

```
header_1 = "ABC123"  
header_2 = "DEF456"  
header_3 = "HIJ789"  
seq_1 = "ATCGTACGATCGATCGCTAGACGTATCG"  
seq_2 = "actgatcgacgatcgatcgatcacgact"  
seq_3 = "ACTGAC-ACTGT--ACTGTA----CATGTG"
```

```
print('>' + header_1 + '\n' + seq_1)  
print('>' + header_2 + '\n' + seq_2)  
print('>' + header_3 + '\n' + seq_3)
```

IZLAZ

```
>ABC123  
ATCGTACGATCGATCGATCGCTAGACGTATCG  
>DEF456  
actgatcgacgatcgatcgatcacgact  
>HIJ789  
ACTGAC-ACTGT--ACTGTA----CATGTG
```

Redosled linija je dobar. Prvi pokušaj sa šest print naredbi zaboravio je znak `>`; jedan print sa `\n` ispisuje i zaglavlje i sekvencu.

Korak 2: sređivanje sekvenci i pisanje u fajl

```
output = open("sequences.fasta", "w")
output.write('>' + header_1 + '\n' + seq_1)
output.write('>' + header_2 + '\n' + seq_2.upper())
output.write('>' + header_3 + '\n' + seq_3.replace('-', ''))
```

```
SEQUENCES.FASTA
>ABC123
ATCGTACGATCGATCGATCGCTAGACGTATCG>DEF456
ACTGATCGACGATCGATCGATCACGACT>HIJ789
ACTGACACTGTACTGTACATGTG
```

`upper()` sređuje drugu sekvencu; zamena crtice praznim stringom "" briše crtice.

Linije su se spojile: `print` sam dodaje novi red, a `write` ne.

Korak 3: novi red na kraju svakog zapisa

```
# header_1..3 i seq_1..3 kao u koraku 1
output = open("sequences.fasta", "w")
# prvi zapis
output.write('>' + header_1 + '\n' + seq_1 + '\n')
# drugi zapis, velikim slovima
output.write('>' + header_2 + '\n' + seq_2.upper() + '\n')
# treći zapis, bez crtica
output.write('>' + header_3 + '\n' + seq_3.replace('-', '') + '\n')
output.close()
```

```
>ABC123
ATCGTACGATCGATCGATCGCTAGACGTATCG
>DEF456
ACTGATCGACGATCGATCGATCACGACT
>HIJ789
ACTGACACTGTACTGTACATGTG
```

Složen argument je niz jednostavnih delova: „znak >, zaglavlje, novi red, sekvenca bez crtica, novi red”.

Jedan fajl po sekvenci

```
output_1 = open(header_1 + ".fasta", "w")
output_2 = open(header_2 + ".fasta", "w")
output_3 = open(header_3 + ".fasta", "w")
output_1.write('>' + header_1 + '\n' + seq_1 + '\n')
output_2.write('>' + header_2 + '\n' + seq_2.upper() + '\n')
output_3.write('>' + header_3 + '\n' + seq_3.replace('-', '') + '\n')
```

Rezultat

ABC123.fasta
DEF456.fasta
HIJ789.fasta

Previše ponavljanja

Četiri dela koda, i svaki ima tri skoro ista reda: zaglavlja, sekvence, otvaranje fajlova i pisanje. To je mnogo kucanja i lako dovodi do greške. Liste i petlje iz poglavlja 4 rešavaju upravo taj problem.

Kodirajući i nekodirajući deo u dva fajla

```
# otvori fajl i pročitaj sadržaj
dna_file = open("genomic_dna.txt")
my_dna = dna_file.read()
# izdvoj delove sekvence
exon1 = my_dna[0:63]
intron = my_dna[63:90]
exon2 = my_dna[90:10000]
# otvori dva izlazna fajla
coding_file = open("coding_dna.txt", "w")
noncoding_file = open("noncoding_dna.txt", "w")
# upiši sekvence
coding_file.write(exon1 + exon2)
noncoding_file.write(intron)
```

Polazimo od rešenja iz poglavlja 2. Šta se menja:

- 1 sekvenca se čita iz fajla, umesto da je upisana u kod
- 2 dva nova fajl objekta za izlaz, otvorena za pisanje
- 3 egzoni idu u jedan fajl, intron u drugi

Rezime i vežbe

Šta smo naučili

- Rad sa fajlom ima dva koraka: prvo open, pa čitanje ili pisanje
- Ime fajla, fajl objekat i sadržaj su tri različite stvari
- `rstrip("\n")` uklanja novi red sa kraja
- Režimi "r", "w" i "a"; metodi write i close
- Putanje se razlikuju po operativnim sistemima
- Nepostojeći fajl daje `FileNotFoundError`

Vežbe

1. Podelite genomsku DNK iz `genomic_dna.txt` na kodirajući i nekodirajući deo i upišite ih u dva fajla
2. Napravite FASTA fajl sa tri sekvence (prethodni slajd)
3. Napravite tri FASTA fajla, po jedan za svaku sekvencu; ime fajla je zaglavlje + `.fasta`

Sledi: čitanje liniju po liniju (pogl. 4), rad sa fajlovima i folderima i imena fajlova koja zadaje korisnik (pogl. 9).